Smoke plume NO_2 pipeline — guide and tuning reference

Smoke plume NO_2 pipeline — a plain-English guide

This page explains **what the code is doing** and **why**, without assuming you speak remote-sensing jargon. **Figures below are stored under `[docs/images/](images/)`** (committed in git) so they show in GitHub, VS Code / Cursor Markdown Preview, and other renderers—open this file from the repo so relative image paths resolve.

For math and symbols, see PROJECT.md. **Every tunable knob** (CLI flags and defaults across the pipeline scripts) lives in pipeline_tuning_parameters.md. A **single PDF or HTML** that merges this guide with that full tuning reference is built by `[scripts/build_pipeline_guide_pdf.py](../scripts/build_pipeline_guide_pdf.py)` (outputs `docs/pipeline_layman_guide.pdf` and `docs/pipeline_layman_guide.html`).

The one-sentence version

We combine a **sharp photo of smoke** (Planet) with a **coarse map of air pollution** (TEMPO NO_2) to estimate how much **extra nitrogen dioxide** is sitting in the plume—stated as a **mass in kilograms**—after subtracting a "normal clean air" level.

Two satellites, two jobs

	Planet (PlanetScope)	TEMPO (NASA, geostationary)
What it's good at	Seeing **where smoke is** on the ground (meters-scale).	Measuring **NO_2 in the column of air** above the ground (kilometers-scale "pixels").
Analogy	A detailed **photo** of the smoke plume.	A **low-resolution weather-style grid** of pollution over a huge area.
Catch	Planet doesn't directly give you "kg of NO_2."	TEMPO doesn't see smoke shape at fence-line detail.

The pipeline's trick is to **merge** them: use Planet to say *how much of each big TEMPO pixel is covered by smoke*, then weight the NO_2 signal accordingly.

The story in five steps (ELI5)

Step 1 — Line up the moment in time

Wildfires move. Satellites pass at different times. We pick a Planet scene and a TEMPO granule that are **close in time** so we're not comparing lunch to dinner. Your `case.json` can record those times under `time_match` (see PROJECT.md).

Step 2 — Paint "smoke" on the Planet image

The computer doesn't "see" smoke the way humans do. It uses **simple spectral rules** on surface reflectance—for example, a **blue vs near-infrared** test— to mark pixels that look like smoke. That produces a **mask**: each Planet pixel is "smoke" or "not smoke" (with invalid areas masked out).

Nothing here is perfect: thin cloud, haze, bright dirt, and real smoke can get confused. That's why there are tuning docs like smoke_mask_challenges.md.

Step 3 — Pour the fine mask into big TEMPO "bins" (f_p)

Imagine pouring sand through a sieve:

- Planet pixels are **tiny**.
- TEMPO pixels are **huge**.

For each big TEMPO cell, we ask: *of the tiny Planet pixels inside, what fraction looked like smoke?* That fraction is called **f_p** (overlap fraction). It's always between **0** and **1**.

If **f_p = 0**, that TEMPO pixel has essentially **no smoke** in it (according to Planet). If **f_p = 0.3**, roughly **30%** of that cell "looks like smoke" at Planet resolution.

Schematic: fine Planet grid averaged into coarse TEMPO cells (f_p)

*Figure: same idea at two scales—the fine "smoke" map is averaged into each big TEMPO pixel to get **f_p**.*

Step 4 — Subtract "normal" NO_2 (background)

The atmosphere always has some NO_2 (cities, chemistry, etc.). We estimate a **background** column using TEMPO pixels that are **clean in the Planet sense** (low **f_p**). Then:

excess column ≈ total column − background

So we focus on **extra** NO_2, not the absolute number on a smoggy day.

Schematic: total column, background, and excess

*Figure: we only keep the **excess** above a background estimated from low-**f_p** pixels.*

Step 5 — Combine and add up to kilograms

For each TEMPO pixel we combine:

plume-weighted excess ≈ f_p × (excess NO_2 column)

Then we **multiply by the geographic size** of each TEMPO pixel and sum over the grid. Unit conversions turn **molecules per cm²** into **kilograms of NO_2**—that's the **total excess NO_2 (kg)** in `pipeline_summary.json`.

Schematic: f_p, excess column, and integration to kg

*Figure: multiply overlap by excess column, then sum over the grid with pixel area to get **kg**.*

Flowchart (technical but readable)

```
flowchart LR
  P[Planet SR GeoTIFF] --> M[Smoke mask on Planet grid]
  T[TEMPO VCD GeoTIFF] --> B[Background VCD from low f_p pixels]
  M --> F[f_p: warp mask to TEMPO grid]
  T --> D[Delta VCD = VCD - VCD_bg]
  F --> X[f_p × Delta VCD per pixel]
  D --> X
  X --> S[Sum × pixel area → kg NO₂]
```

Six case studies — same pipeline, six real fires

You have **six folders** under `smoke-plume-data/` (airport, bridge, eaton, line, palisades, park). Think of them as **six homework problems with the same instructions**: same mask recipe, same background rule, same mass recipe—but **different days, fires, scenes, and TEMPO granules**.

The mental model (so it "sticks")

Layer	Plain English	What varies across the six
Planet	"Where does it *look* smoky at fine scale?"	Scene extent, sun angle, haze, terrain—mask is never identical.
TEMPO	"How much NO_2 is in the air column above each big pixel?"	Chemistry, transport, granule time, and **how huge the warped grid is**.
f_p	"When we pour Planet into TEMPO, how smoky is each big pixel?"	Same math; different overlap patterns.
Mass (kg)	Sum of **f_p × excess column × pixel area** over the **whole GeoTIFF**.	**Not** "tons from the fire engine"—it's **integrated over the entire raster domain** unless you clip.

So: treat this as *one consistent measurement machine pointed at six different worlds*, not six guaranteed-comparable "fire scores."

Numbers for all six (one batch, same defaults)

These match the study batch documented in STUDY_BATCH_RESULTS.md.

Region	Total excess NO_2 (kg)	"Smoky" TEMPO pixels (f_p > 0.01)	TEMPO grid (pixels)	VCD background median (molec/cm²)
airport	~1,540	470	~9.31 M	~3.8×10^{14}
bridge	~4,001	532	~8.55 M	~5.0×10^{14}
eaton	~1,702	840	~7.94 M	~5.7×10^{14}
line	~2,769	433	~7.95 M	~6.8×10^{14}
palisades	~1,467	901	~8.92 M	~6.5×10^{14}
park	~1,486	662	~9.68 M	~3.2×10^{14}

Notice: TEMPO **grid sizes** (pixel counts) vary by **granule coverage and warp resolution**—check the table column before comparing total kg across regions. Mass always runs over **whatever pixels exist in that GeoTIFF**.

Cross-region comparison plots (from the batch summaries)

These charts are built by `scripts/render_case_study_comparison.py` and saved next to this guide as PNGs. They help you compare regions **without** treating the table as a simple leaderboard.

1 — Total kg, plume-pixel count, and grid size

Bar charts: mass, plume pixels, grid size for six regions

2 — Mass vs. count of "plume-ish" pixels

If high mass always came from "more plume pixels," points would hug a neat line. They don't—**a few pixels with huge excess column** can rival **many pixels with modest excess**.

Scatter: mass vs plume pixel count

3 — Rough "kg per plume-ish pixel" (ranked)

This is **not** a physical emission factor—it's a **cartoon ratio** for intuition (Palisades often stands out when **few** cells carry **most** of the signal).

Bar: kg per plume pixel (ranked)

Case-by-case maps (cropped previews)

For **each** study region, the pipeline exports **cropped** PNGs next to the plume: **f_p** (how smoky each big TEMPO cell is) and **f_p × ΔNO₂** (plume-weighted excess). Below that, **histograms** show how **f_p** and plume ΔNO_2 are distributed when **f_p > 0.01**. These copies live under `[docs/images/cases/](images/cases/)` so you can read this guide offline or on GitHub; refresh them after regenerating maps (see Commands).

Airport

Plume overlap f_p	**Plume-weighted excess f_p × ΔNO₂**
airport f_p	airport plume delta

airport histograms

~1,540 kg total excess NO_2; 470 coarse pixels with f_p > 0.01 (see table above).

Bridge

Note: Bridge needed **Planet surface reflectance (SR)** in `case.json`; the wrong product can drive **f_p** (and mass) to ~0 even when the atmosphere isn't clean—see STUDY_BATCH_RESULTS.md.

Plume overlap f_p	**Plume-weighted excess f_p × ΔNO₂**
bridge f_p	bridge plume delta

bridge histograms

~4,001 kg; 532 plume-ish pixels after the SR fix.

Eaton

Plume overlap f_p	**Plume-weighted excess f_p × ΔNO₂**
eaton f_p	eaton plume delta

eaton histograms

*~1,702 kg; **840** plume-ish pixels—many flagged cells, not necessarily the highest total kg.*

Line

Plume overlap f_p	Plume-weighted excess f_p × ΔNO_2
line f_p	line plume delta

line histograms

*~2,769 kg with **433** plume-ish pixels—**column strength** can outweigh raw pixel count.*

Palisades

Plume overlap f_p	Plume-weighted excess f_p × ΔNO_2
palisades f_p	palisades plume delta

palisades histograms

*~1,467 kg with **901** plume-ish pixels in this batch—**signal per cell** and **swath footprint** still matter more than the row in the table alone.*

Park

Plume overlap f_p	Plume-weighted excess f_p × ΔNO_2
park f_p	park plume delta

park histograms

~1,486 kg; 662 plume-ish pixels.

ELI5 contrasts (scientifically honest)

- **Line vs. eaton:** Line can show **higher total kg** with **fewer** "smoky" coarse pixels than eaton—because **column strength** matters as much as **how many** cells pass the f_p threshold.
- **Palisades vs. park:** Total kg can land in a **similar band** with **different** plume-pixel counts—**column strength** and **where** f_p lines up with ΔVCD still dominate over a simple pixel tally.
- **Bridge:** After switching to **surface reflectance**, the mask finally "sees" smoke on the TEMPO grid; before that, the pipeline could report **~0** not because the atmosphere was clean, but because **the wrong Planet product** broke the mask.

Statistical-ish vocabulary in human language

- **Central tendency:** Across these six, total kg spans **roughly ~1.5k–4k** with this setup—useful as an **order-of-magnitude family**, not a universal law.
- **Outlier sensitivity:** One bad asset (wrong Planet type) can move a case from **~0** to **thousands of kg**—always check **inputs** before interpreting **outputs**.
- **Correlation ≠ causation:** Higher kg does **not** mean "worse fire" in any moral or regulatory sense; it means **this integrated model** assigned more plume-weighted excess column **inside that GeoTIFF**.

Where this lives in your repo (outputs)

After you run the pipeline (and optionally map exports), each case folder has:

File	Plain English
`pipeline_summary.json`	Numbers: background, pixel counts, **total kg**.
`pipeline_table.csv`	Same headline numbers in a tiny table.
`f_p.tif`	Map of **overlap fractions** on the TEMPO grid.
`delta_vcd.tif`	Map of **excess column** before smoke weighting.
`delta_vcd_plume.tif`	Map of **f_p × excess** (what gets integrated to mass).

Preview PNGs (from `smoke_plume_sanity_check.py` / `study_batch_visuals.py`) live under `maps/`:

- `**f_p_preview.png**` — quick look at where the plume overlaps TEMPO (cropped).
- `**delta_vcd_plume_preview.png**` — where the **weighted** excess is strongest (cropped).
- `**histograms.png**` — how **f_p** and plume ΔVCD are distributed.

Full-resolution GeoTIFFs under `results/` are often **gitignored**. Generate them locally with `scripts/study_batch_visuals.py` (see STUDY_BATCH_RESULTS.md); use `**scripts/sync_guide_case_images.py**` to refresh the **committed** copies in `docs/images/cases/` for this guide.

What the numbers do *not* automatically mean

- **Not a legal emissions inventory.** The mask is heuristic; the domain is often a **whole TEMPO swath**, not a tiny fire perimeter unless you clip.
- **Not "only the fire."** Any smoke-like pixel **anywhere** in the grid contributes.
- **Cross-fire comparisons are delicate.** Different dates, granules, and scene sizes change the story.

Glossary (quick)

Term	Meaning
VCD (vertical column density)	Total NO_2 integrated through the air column above a spot, reported per unit ground area (e.g. molecules per cm^2).
f_p	Fraction of a **TEMPO** pixel that Planet-classified as smoke (0–1).
Background	Typical "clean-ish" column estimated from **low-f_p** TEMPO pixels.
ΔVCD / excess column	Column minus background.
Surface reflectance (SR)	Planet product where lighting is partly normalized so spectra are comparable—use **SR** for the mask unless you know what you're doing.

Commands (if you want to reproduce visuals)

```
# Regenerate GeoTIFF maps + PNG previews for every study case
py -3 scripts/study_batch_visuals.py --results-root results/study_batch

# Copy map PNGs into docs/images/cases/ for this guide (committed assets)
py -3 scripts/sync_guide_case_images.py
```

```
# Rebuild only the small schematic images in this guide
py -3 scripts/render_pipeline_guide_assets.py

# Rebuild the six-case comparison bar/scatter figures (reads pipeline_summary.json when present)
py -3 scripts/render_case_study_comparison.py

# Combined HTML + PDF: this guide + full pipeline_tuning_parameters.md appendix (needs pandoc; PDF
         via Chrome headless or --pdf-engine)
py -3 scripts/build_pipeline_guide_pdf.py
```

Where to go next

Goal	Document
Deeper methodology + pilot metadata	PROJECT.md
Lab notebook style, step checkpoints	results/walkthrough.md
Mask pitfalls and naming	smoke_mask_challenges.md
CLI parameters explained	pipeline_tuning_parameters.md
Batch study table + visual paths	STUDY_BATCH_RESULTS.md

Appendix — Pipeline tuning parameters (full reference)

This appendix is the same content as `docs/pipeline_tuning_parameters.md` (end-to-end knobs).

This document lists **knobs that change numerical outputs** if you adjust them: smoke fraction (f_p), background (VCD_{bg}), excess column, and reported mass. Use it like a checklist of "sliders" when reproducing or sensitivity-testing a run.

Related: smoke_mask_challenges.md (why masks are hard, not more parameters).

Flow (dependency order)

1. **TEMPO L2 → GeoTIFF** — `scripts/tempo_l2_to_4326.py` (VCD grid, QA, optional AMF plume adjustment).
2. **Planet mask / (f_p)** — `scripts/smoke_plume_pipeline.py` (and optional QGIS exports: `planet_smoke_mask_qgis.py`, `export_planet_smoke_step2.py`).
3. **Mass from column** — `scripts/column_to_mass.py` (re-integrates `delta_vcd_plume.tif` if you change units or swap rasters).

Changing an **earlier** step's outputs (especially TEMPO VCD or Planet scene) propagates to everything downstream.

---

1. `tempo_l2_to_4326.py` (TEMPO swath → EPSG:4326)

Parameter	Default	Effect if changed
`--nc`	*(required)*	Different granule → different scene, time, geometry, QA, and VCD field.
`-o` / `--output`	next to `.nc`	Where the warped GeoTIFF is written; downstream scripts point here.
`--bbox` W S E N	full valid swath	**Clips** extent and which pixels are interpolated; changes every TEMPO pixel value and overlap with Planet.
`--res`	~long side / 3600°, min 0.005°	**Finer** res → more pixels, different interpolation; **coarser** → smoother fields, different sub-pixel overlap with (f_p).
`--no-qa`	off (QA on)	If set, keeps pixels with `main_data_quality_flag != 0` → usually **higher** noise / different VCD.
`--max-cloud` F	**0.2**	Masks where `eff_cloud_fraction > F` (TEMPO trace-gas user guide). Stricter F → **fewer** pixels. **`--no-cloud-mask`** skips cloud masking.
`--ground-qa-zero`	off	Also requires `ground_pixel_quality_flag == 0`; can remove more pixels (check ATBD semantics).
`--stack`	off	Writes **multi-band** GeoTIFF (VCD + AMF, cloud, QA…). Affects **`--tempo-vcd-band`** in the main pipeline (band 1 = VCD if stacked as expected).
`--amf-plume-adjust`	off	Rescales tropospheric VCD using scattering weights vs assumed vertical plume shape → **systematic** VCD change where enabled.
`--plume-height-agl-m`	`1000`	Center height (m AGL) of Gaussian plume used in AMF adjustment → changes rescaling.
`--plume-fwhm-m`	`500`	Vertical FWHM (m) of that Gaussian → changes how sensitivity is weighted in height.

Implementation note: `tempo_amf_plume_adjust.py` uses fixed atmosphere approximations (scale height, layer construction); those are **not** CLI parameters but affect results when `--amf-plume-adjust` is on.

---

2. `smoke_plume_pipeline.py` (mask → (f_p) → (VCD_{bg}) → ΔVCD → mass)

Inputs (largest leverage)

Parameter	Default	Effect
`--planet`	Pilot path under `smoke-plume-data/palisades/`	**Different scene / geometry** → new smoke mask and (f_p).
`--tempo`	Pilot warped GeoTIFF	**Different VCD grid** → new background, ΔVCD, mass.
`--out`	`results/smoke_plume`	Output folder only (does not change math).

Vertical column interpretation

Parameter	Default	Effect
`--vcd-units`	`molec_cm2`	Must match the TEMPO GeoTIFF; wrong choice → **wrong mass scaling** (same formula family, different conversion).
`--tempo-vcd-band`	`1`	Which band is read as tropospheric VCD. **Must** match how `tempo_l2_to_4326.py` wrote the file (band 1 = VCD; other bands if `--stack`).

Smoke mask and (f_p)

Parameter	Default	Effect
`--mask-method`	`blue_nir`	Switches rule: `blue_nir`, `ndhi` (G–B / G+B), or `ndhi_bnir` (B–NIR / B+NIR) → different mask and (f_p).
`--blue-nir-max`	`0.42`	For `blue_nir`: smoke where B/NIR **below** this → **strong** sensitivity of plume area.
`--ndhi-smoke-below`	`0.0`	For `ndhi`: smoke where green–blue NDHI **below** this.
`--ndhi-bnir-smoke-above`	`-0.15`	For `ndhi_bnir`: smoke where (B−NIR)/(B+NIR) **above** this.
`--mask-nodata`	`-9999`	Invalid mask pixels for warp; should match GeoTIFF nodata. Changing without consistent files can affect (f_p) averaging at edges.
`--blue-band`	`2`	Wrong band → wrong reflectances and mask.
`--nir-band`	`8`	Same (Planet 8-band SR convention).
`--green-band`	`3`	Used for `--mask-method ndhi` only.

Background (VCD_{bg}) and excess column

Parameter	Default	Effect
`--fp-bg-max`	`0.02`	Max (f_p) for pixels eligible for **background median**. Higher → more “smoky” pixels can enter (VCD_{bg}) → **lower** typical (VCD_{bg}) and **larger** ΔVCD (and mass). Uses **all finite VCD** pixels (negative columns allowed;

Parameter	Default	Effect
		masking is from the GeoTIFF produced in step 1).

Outputs

Flag	Effect
`--write-maps`	Writes `f_p.tif`, `delta_vcd.tif`, `delta_vcd_plume.tif`; mass in JSON is still computed either way when the script runs.

Internal fallbacks (not CLI — change code to tune)

These alter (VCD_{bg}) when there are **few** low-(f_p) pixels:

- If count of `f_p <= fp_bg_max` is **< 50**, the pipeline relaxes to `f_p <= min(0.05, fp_bg_max + 0.03)`.
- If still **< 10**, it uses the **15th percentile** of valid VCD over all valid pixels instead of the median over low-(f_p) pixels.

Summary stats only (not mass): `pixels_plume_fp_gt_0.01` uses a fixed **`f_p > 0.01`** threshold in code.

---

3. `export_planet_smoke_step2.py` (Step 2 checkpoint)

Same **mask-related** parameters as the main pipeline for Planet exports (`--mask-method`, `--blue-nir-max`, `--ndhi-*`, bands, `--mask-nodata`) plus **`--preview-max-side`** (preview resolution only — does not change full-res mask or (f_p) written at full resolution).

Parameter	Effect
`--planet`, `--tempo`, `--out`	Paths; different inputs → different outputs.

---

4. `planet_smoke_mask_qgis.py` (QGIS-oriented exports)

Same mask tunables as §2–3. Additional:

Parameter	Effect
`--preview-max-side`	Preview size only.
`--no-preview` / `--mask-only`	What files are written; full-res mask unchanged if produced.
`--with-fp`	Computes `f_p_tempo_grid.tif` using same warp as pipeline when `--tempo` is set.

---

5. `column_to_mass.py` (Step 5 re-mass from raster)

Parameter	Default	Effect
`--raster`	step_04 plume raster	**Different GeoTIFF →** different integrated mass (e.g. after regenerating `delta_vcd_plume.tif`).

Parameter	Default	Effect
`--vcd-units`	`molec_cm2`	Must match raster units; wrong value → **wrong kg**.
`--out-json`	`results/step_05_mass/column_mass.json`	Output path only.

Constants **`AVOGADRO`** and **`M_NO2_KG_PER_MOL`** in `smoke_plume_pipeline.py` define the conversion; change only for a deliberate unit/molar-mass revision.

---

6. Other scripts (minor / diagnostic)

Script	Tunables
`compare_ratio_nd_smoke_mask.py`	`--blue-nir-max`, `--blue-band`, `--nir-band`, `--planet` — comparison only.
`smoke_plume_sanity_check.py`	`--results-dir` — reads existing outputs; does not change pipeline numbers.

---

7. Batch and study-batch helpers

`run_all_cases.py` (many regions, one CLI)

Runs `smoke_plume_pipeline.run(...)` once per discovered case. **All pipeline numeric flags** mirror the main pipeline (same defaults as §2): `--vcd-units`, `--mask-method`, `--blue-nir-max`, `--ndhi-smoke-below`, `--ndhi-bnir-smoke-above`, `--mask-nodata`, `--fp-bg-max`, `--blue-band`, `--nir-band`, `--green-band`, `--tempo-vcd-band`, `--write-maps`.

Parameter	Default	Effect
`--cases-root`	*(optional if manifest set)*	Folder with one subfolder per region (`case.json` or `planet.tif` + `tempo.tif`).
`--manifest`	off	JSON list of cases (see `cases_manifest.example.json`); alternative to `--cases-root`.
`--out-root`	*(required)*	Base output; each case writes to `out-root/<case_id>/`.
`--dry-run`	off	List cases only; no pipeline runs.
`--fail-fast`	off	Stop on first failure; default is run all and record errors in `batch_summary.json`.

Case-specific inputs come from each folder's `case.json` (paths, optional `time_match`), not from extra CLI knobs.

`study_batch_visuals.py` (maps + PNG previews after the fact)

Re-runs the pipeline with parameters **read from** each case's `pipeline_summary.json`, then runs `smoke_plume_sanity_check.py`. Does not introduce new **math** parameters beyond what is already stored in that JSON.

Parameter	Default	Effect
`--results-root`	`results/study_batch`	Parent of per-case folders (`airport`, `bridge`, …).
`--only`	off	Space-separated case ids; if omitted, every case with `pipeline_summary.json` is processed.

`smoke_plume_sanity_check.py`

Parameter	Default	Effect
`--results-dir`	pilot results path	Folder with pipeline outputs; writes `maps/*.png`, `sanity_report.json`. Read-only for VCD/mass numbers unless you re-run the pipeline first.

Quick “sensitivity” order (what to move first)

1. **Inputs:** TEMPO granule / warping (`--bbox`, `--res`, QA, cloud, **`--amf-plume-adjust`** and height/FWHM).
2. **Planet scene** and **mask method** + thresholds (`--blue-nir-max`, `--ndhi-*`, `--mask-method`).
3. **Background** (`--fp-bg-max`).
4. **Band index** mistakes (`--tempo-vcd-band`, Planet bands).
5. **`--vcd-units`** anywhere VCD is interpreted (pipeline + `column_to_mass.py`).

Recording **`pipeline_summary.json`** after each run captures most CLI-relevant parameters for the main pipeline step.